Understanding the Transformer Architecture

Building an Encoder-Decoder from Scratch in PyTorch

Based on *Attention Is All You Need* (Vaswani et al., 2017)

Sawan

May 15, 2026

Abstract

This report documents what I learned while implementing the Transformer architecture from scratch in PyTorch. I break down every component of the model — multi-head attention, positional encoding, the encoder-decoder structure — and explain the math behind each piece. The goal was to move past surface-level understanding and actually build the thing, layer by layer, to see how it all fits together.

Contents

1 Introduction

Before 2017, sequence-to-sequence models were dominated by recurrent neural networks (RNNs) and their variants — LSTMs and GRUs. These models processed tokens one at a time, left to right. That sequential bottleneck made them slow to train and limited their ability to capture long-range dependencies.

The Transformer changed everything. Introduced by Vaswani et al. in *Attention Is All You Need* [1], it threw out recurrence entirely and replaced it with a mechanism called **self-attention**. The result was a model that could look at an entire sequence at once, in parallel, and learn which parts matter for each prediction.

Today, every major language model — GPT, BERT, T5, LLaMA — is built on the Transformer. Understanding it is no longer optional if you want to work in deep learning.

This report covers:

- The key contributions of the original paper
- Every component of the architecture, with math
- Implementation details from building it in PyTorch
- What I actually learned from doing this

2 The "Attention Is All You Need" Paper

2.1 Context and Motivation

In 2017, the state of the art for machine translation was encoder-decoder models using LSTMs with attention (Bahdanau et al., 2015). These models worked, but they had two major problems:

1. **Sequential computation.** RNNs process tokens in order. You can't compute the representation of token t until you've processed tokens 1 through $t-1$. This kills parallelism and makes training slow on GPUs.
2. **Long-range dependencies.** Even with LSTMs, information from early tokens gets diluted as it passes through many timesteps. The model struggles to connect a word at position 5 with a word at position 80.

The Transformer's core insight: *you don't need recurrence at all.* Attention alone is enough to model dependencies between any two positions, regardless of distance, and it can be computed entirely in parallel.

2.2 Key Contributions

The paper made several contributions that are worth spelling out:

- **Self-attention as the primary mechanism.** Every position attends to every other position directly. No recurrence, no convolutions.
- **Multi-head attention.** Instead of one attention function, run h attention heads in parallel, each learning different relationships.
- **Positional encoding.** Since there's no recurrence, the model has no notion of order. Fixed sinusoidal encodings inject position information.
- **Scalability.** The architecture trains significantly faster than RNN-based models because everything runs in parallel.

The model achieved state-of-the-art BLEU scores on English-to-German and English-to-French translation benchmarks, while training in a fraction of the time.

3 Architecture Deep Dive

The Transformer follows an encoder-decoder structure. The encoder reads the input sequence and produces a sequence of continuous representations. The decoder generates the output sequence one token at a time, attending to both its own previous outputs and the encoder's representations.

3.1 Scaled Dot-Product Attention

This is the fundamental building block. Given queries Q, keys K, and values V, attention computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{1}$$

where d_k is the dimension of the keys. The scaling factor $\sqrt{d_k}$ prevents the dot products from growing too large, which would push the softmax into regions with vanishingly small gradients.

Intuitively: for each query, compute a similarity score against every key, normalize those scores into a probability distribution, then take a weighted sum of the values.

3.2 Multi-Head Attention

Instead of performing a single attention function with d_{model}-dimensional keys, queries, and values, the paper projects them into h different subspaces and runs attention in parallel:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O \tag{2}$$

$$\text{where head}_i = \text{Attention}(QW_i^Q,\, KW_i^K,\, VW_i^V) \tag{3}$$

Each head operates on a subspace of dimension $d_k = d_{\text{model}}/h$. This lets the model jointly attend to information from different representation subspaces at different positions. A single attention head can't do this — it averages and loses information.

3.3 Positional Encoding

Since the Transformer processes all positions in parallel, it has no inherent sense of token order. Positional encodings are added to the input embeddings to inject this information:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \tag{4}$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \tag{5}$$

The frequencies form a geometric progression from 2π to $10000 \cdot 2\pi$. This design has a nice property: for any fixed offset k, PE_{pos+k} can be represented as a linear function of PE_{pos}, which the paper hypothesized would help the model learn relative positions.

3.4 Position-Wise Feed-Forward Network

Each layer contains a fully connected feed-forward network applied to each position independently:

$$\mathrm{FFN}(x) = \mathrm{ReLU}(xW_1 + b_1)\, W_2 + b_2 \tag{6}$$

The inner dimension $d_{ff} = 2048$ is four times the model dimension $d_{\text{model}} = 512$. This expansion-contraction pattern gives the network capacity to learn complex transformations at each position.

3.5 Residual Connections and Layer Normalization

Every sub-layer (attention, FFN) is wrapped with a residual connection followed by layer normalization:

$$\text{output} = \mathrm{LayerNorm}(x + \mathrm{SubLayer}(x)) \tag{7}$$

Residual connections let gradients flow directly through the network, making deep models trainable. Layer normalization stabilizes the activations, preventing them from blowing up or vanishing across layers.

3.6 The Encoder

The encoder is a stack of $N = 6$ identical layers. Each layer has:

1. Multi-head **self-attention** (each position attends to all positions in the input)
2. Position-wise **feed-forward** network

Both sub-layers use residual connections and layer normalization.

3.7 The Decoder

The decoder is also a stack of $N = 6$ identical layers, but each layer has three sub-layers:

1. **Masked** multi-head self-attention (prevents attending to future positions)
2. Multi-head **cross-attention** (attends to encoder output)
3. Position-wise **feed-forward** network

The masking in the first sub-layer is crucial: during training, we feed the entire target sequence at once, but we can't let position i see positions $i+1, i+2, \ldots$. A causal mask (upper-triangular matrix of $-\infty$) handles this.

3.8 Output Layer

The decoder output is projected through a linear layer to produce logits over the target vocabulary, followed by a softmax to get probabilities.

4 Implementation Details

I built the Transformer in PyTorch, splitting each component into its own module:

A few implementation choices worth noting:

File	Component
`attention.py`	Scaled dot-product + multi-head attention
`embeddings.py`	Token embedding + sinusoidal positional encoding
`feed_forward.py`	Position-wise feed-forward network
`masks.py`	Padding mask + causal mask
`encoder.py`	Encoder layer + encoder stack
`decoder.py`	Decoder layer + decoder stack
`transformer.py`	Full model (encoder + decoder + output projection)

Table 1: Module breakdown

- **Reshaping for multi-head attention.** Instead of creating separate W^Q, W^K, W^V matrices for each head, I use a single `nn.Linear(d_model, d_model)` and reshape the output into `(batch, heads, seq_len, d_k)`. This is more efficient and equivalent.
- **Mask broadcasting.** The padding mask has shape `(batch, 1, 1, seq_len)` and the causal mask has shape `(1, 1, seq_len, seq_len)`. PyTorch's broadcasting handles combining them without explicit expansion.
- **Positional encoding as a buffer.** I use `register_buffer` so the PE tensor moves to the correct device with the model but isn't treated as a trainable parameter.
- **Weight sharing.** The original paper ties the embedding weights with the output projection. I kept them separate for clarity.

5 What I Learned

Building the Transformer from scratch taught me things that reading the paper alone never could:

1. **Attention is just weighted averaging.** Strip away the notation and attention is: compute similarity scores, normalize them, take a weighted sum. The power comes from *learning* what to attend to through the W^Q, W^K, W^V projections.

2. **The scaling factor matters.** Without dividing by $\sqrt{d_k}$, the dot products grow proportionally with dimension, pushing softmax outputs toward one-hot vectors. This means almost zero gradient for most positions, and training stalls. I saw this firsthand when I forgot the scaling in an early version.

3. **Multi-head attention is about diversity.** One attention head computes one weighted average per position. That's limiting. Multiple heads let different heads specialize — one might track syntactic relationships, another semantic ones. The concatenation preserves all of this.

4. **Positional encoding is surprisingly elegant.** The sinusoidal approach doesn't require learning anything, scales to arbitrary sequence lengths, and the linear relationship between positions at different offsets gives the model a way to reason about relative positions.

5. **Residual connections are essential, not optional.** Without them, a 6-layer Transformer is basically untrainable. Gradients vanish before reaching the early layers. Adding x back at each sub-layer provides a direct gradient highway.

6. **The mask logic is tricky.** Getting the shapes right for padding masks vs. causal masks, and understanding when to apply each one, was the most debugging-intensive part of the implementation. The decoder needs both simultaneously.

7. **The architecture is simpler than it looks.** Once you break it into pieces — attention, FFN, norm, residual — each piece is straightforward. The elegance is in how they compose.

6 Conclusion

The Transformer is one of those rare architectures that is both simple in concept and profound in impact. It replaced recurrence with attention, enabling massive parallelism and better modeling of long-range dependencies. Every modern large language model is a descendant of this design.

Building it from scratch was the right way to learn it. Papers give you equations; code gives you understanding. I now have a solid mental model of how queries, keys, and values flow through the network, how masking prevents information leakage, and why each design choice was made.

If you're studying deep learning and haven't built a Transformer by hand, I'd recommend it. It's one of those exercises where the effort pays for itself many times over.

References

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems (NeurIPS), 30. https://arxiv.org/abs/1706.03762

[2] Bahdanau, D., Cho, K., & Bengio, Y. (2015). *Neural Machine Translation by Jointly Learning to Align and Translate*. International Conference on Learning Representations (ICLR). https://arxiv.org/abs/1409.0473

[3] Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). *Layer Normalization*. https://arxiv.org/abs/1607.06450

[4] He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. IEEE Conference on Computer Vision and Pattern Recognition (CVPR). https://arxiv.org/abs/1512.03385